Laboratory report of Nanjing University of Information Science and Technology

Experiment Title Cloud Computing
Data 2026/4/16 Class AI 1
Name Weijie Xu ID 202383910029

一、Experimental purpose

(1) To understand the fundamental architecture and workflow of Istio service mesh.This includes gaining insight into the overall structure of Istio, particularly the interaction between the control plane and the data plane, as well as how sidecar proxies enable transparent communication between services.
(2) To learn the installation and configuration process of Istio in a Kubernetes cluster.This involves performing a complete deployment of Istio using standard tools such as istioctl, and configuring the cluster environment to support service mesh functionalities.
(3) To explore automatic sidecar injection and its role in traffic interception.This step focuses on understanding how Istio automatically injects Envoy sidecar proxies into application pods, and how these proxies intercept all inbound and outbound traffic.
(4) To verify internal service communication within a microservices architecture.The experiment examines how services communicate with each other inside the Kubernetes cluster under the control of Istio.
(5) To implement external access using Kubernetes Gateway API. This part aims to configure and deploy a Gateway resource to expose internal services to external clients.
(6) To observe and analyze service interactions using visualization tools such as Kiali. By generating traffic and inspecting the resulting service graph, this step focuses on analyzing real-time interactions between services.

二、Experimental Contents

- **Environment preparation and Kubernetes cluster verification**

The experiment began with preparing the runtime environment and ensuring that a functional Kubernetes cluster was available. Basic commands such as kubectl cluster-info and kubectl get nodes were executed to verify cluster status, node availability, and network connectivity, providing a stable foundation for subsequent deployment.

- **Istio installation and initial configuration**

Istio was installed using the istioctl tool with a predefined demo configuration profile. The installation process included setting up the control plane components and

verifying their operational status. In addition, the system PATH was configured to ensure that Istio command-line tools could be accessed globally.

- **Namespace configuration and sidecar injection setup**

The default namespace was labeled to enable automatic sidecar injection. This step ensured that all subsequently deployed application pods would automatically include Envoy proxies, allowing Istio to intercept and manage traffic without requiring manual modification of application code.

- **Deployment of the Bookinfo microservices application**

The Bookinfo sample application was deployed to simulate a typical microservices architecture. Multiple services such as productpage, reviews, ratings, and details were created, forming a multi-tier service structure. Pod and service statuses were monitored to ensure successful deployment and proper integration with Istio.

- **Internal service communication verification**

Service connectivity within the cluster was tested by issuing requests from one service pod to another. The response results were examined to confirm that service discovery and routing mechanisms were functioning correctly under the Istio service mesh environment.

- **Installation of Gateway API CRDs and external access configuration**

Gateway API custom resource definitions were installed to support modern ingress management. A Gateway resource was then configured to expose the Bookinfo application, enabling external access through a unified entry point and demonstrating how traffic is routed into the cluster.

- **Access validation through port forwarding**

Port forwarding was used to map the gateway service to a local port, allowing the application to be accessed via a browser. The successful rendering of the Bookinfo interface confirmed that external traffic could reach internal services correctly through the configured gateway.

- **Visualization and traffic analysis using Kiali**

The Kiali dashboard was deployed and accessed to visualize service interactions. By generating multiple requests to the application, traffic flow and service dependencies were observed in real time, providing an intuitive understanding of how requests propagate within the service mesh.

三、Detailed steps of the experiment

Minikube is started to create a local Kubernetes cluster on the machine, which sets up the control plane and necessary system components inside a Docker container. Although there is a warning about failing to access the external image registry, the cluster still starts successfully using available resources.

Then, the cluster status is checked, showing that the Kubernetes control plane and CoreDNS service are running normally. This means the environment is ready for deploying applications or continuing with Istio installation.

```
(base) PS C:\Users\nxdxwj> minikube start
😄 Microsoft Windows 11 Pro 25H2 上的 minikube v1.38.1
✨ 根据现有的配置文件使用 docker 驱动程序
👍 在集群中 "minikube" 启动节点 "minikube" primary control-plane
🚜 正在拉取基础镜像 v0.0.50 ...
🔄 正在为"minikube"重启现有的 docker container ...-
❗ 从 Minikube 的 container 内部连接到 https://registry.k8s.io/ 失败
💡 要获取新的外部镜像，可能需要配置代理：https://minikube.sigs.k8s.io/docs/reference/networking/proxy/
🐳 正在 Docker 29.2.1 中准备 Kubernetes v1.35.1…|
🔎 正在验证 Kubernetes 组件...
    ▪ 正在使用镜像 gcr.io/k8s-minikube/storage-provisioner:v5
🌟 启用插件： storage-provisioner, default-storageclass
🏄 完成！ kubectl 现在已配置，默认使用"minikube"集群和"default"命名空间
```

```
(base) PS C:\Users\nxdxwj> kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:43767
CoreDNS is running at https://127.0.0.1:43767/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
```

```
(base) PS C:\Users\nxdxwj> kubectl version
Client Version: v1.34.1
Kustomize Version: v5.7.1
Server Version: v1.35.1
```

Istio has been successfully downloaded and extracted from the official repository. The system also indicates the location of the installed files and reminds you to add the bin directory to the environment path so that the istioctl tool can be used globally.

```
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/lab04$ curl -L https://istio.io/downloadIstio | sh
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--   0     0    0     0    0     0      0
    0 --:--:-- --:--:-- --:--:-- 100   102  100   102    0     0    122      0 --:--:-- --:--:-- --:--:--   122
  0  5124    0     0    0     0      0      0 --:--:--  0:00:01 --:--:-- 100  5124  100  5124    0     0   2884
    0  0:00:01  0:00:01 --:--:-- 1000k

Downloading istio-1.29.2 from https://github.com/istio/istio/releases/download/1.29.2/istio-1.29.2-linux-amd64.ta
r.gz ...

Istio 1.29.2 download complete!

The Istio release archive has been downloaded to the istio-1.29.2 directory.

To configure the istioctl client tool for your workstation,
add the /home/ubuntu/cloud_lab/gitea-lab/lab04/istio-1.29.2/bin directory to your environment path variable with:
         export PATH="$PATH:/home/ubuntu/cloud_lab/gitea-lab/lab04/istio-1.29.2/bin"
```

The istioctl tool is working correctly, but Istio has not been installed in the Kubernetes cluster yet. The message indicates that there are no running Istio components in the istio-system namespace.

```
(base) PS D:\我的\大三\大三下\云计算导论\实验\石磊实验\Code\lab04> istioctl version
no ready Istio pods in "istio-system"                    "
1.22.3
```

Istio has been successfully installed into the Kubernetes cluster using the demo configuration. Core components such as the control plane and ingress/egress gateways are now running, which means the service mesh environment is ready for use. The warning only indicates that the version may be outdated, but it does not affect the current setup.

```
(base) PS D:\我的\大三\大三下\云计算导论\实验\石磊实验\Code\lab04> istioctl install --set profile=
demo -y
WARNING: Istio 1.22.0 may be out of support (EOL) already: see https://istio.io/latest/docs/releas
es/supported-releases/ for supported releases
✓ Istio core installed
✓ Istiod installed
✓ Egress gateways installed
✓ Ingress gateways installed
✓ Installation complete                                                                          M
ade this installation the default for injection and validation.
```

Gateway API has been successfully installed into the Kubernetes cluster. The output shows that multiple Custom Resource Definitions, such as gateways and routes, have been created, which are required for managing traffic routing in Istio.

```
(base) PS D:\我的\大三\大三下\云计算导论\实验\石磊实验\Code\lab04> kubectl apply --server-side -f
"https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.4.0/experimental-install.yaml
"
customresourcedefinition.apiextensions.k8s.io/backendtlspolicies.gateway.networking.k8s.io servers
ide-applied
customresourcedefinition.apiextensions.k8s.io/gatewayclasses.gateway.networking.k8s.io serverside-
applied
customresourcedefinition.apiextensions.k8s.io/gateways.gateway.networking.k8s.io serverside-applie
d
customresourcedefinition.apiextensions.k8s.io/grpcroutes.gateway.networking.k8s.io serverside-appl
ied
customresourcedefinition.apiextensions.k8s.io/httproutes.gateway.networking.k8s.io serverside-appl
ied
customresourcedefinition.apiextensions.k8s.io/referencegrants.gateway.networking.k8s.io serverside
-applied
customresourcedefinition.apiextensions.k8s.io/tcproutes.gateway.networking.k8s.io serverside-appli
ed
customresourcedefinition.apiextensions.k8s.io/tlsroutes.gateway.networking.k8s.io serverside-appli
ed
```

The Bookinfo microservices application has been successfully deployed into the Kubernetes cluster. Multiple services, deployments, and service accounts are created, indicating that all components of the application are now running and ready for further testing and access.

```
(base) PS D:\我的\大三\大三下\云计算导论\实验\石磊实验\Code\lab04> cd .\istio-1.22.3
(base) PS D:\我的\大三\大三下\云计算导论\实验\石磊实验\Code\lab04\istio-1.22.3> kubectl apply -f s
amples/bookinfo/platform/kube/bookinfo.yaml
service/details created
serviceaccount/bookinfo-details created
deployment.apps/details-v1 created
service/ratings created
serviceaccount/bookinfo-ratings created
deployment.apps/ratings-v1 created
service/reviews created
serviceaccount/bookinfo-reviews created
deployment.apps/reviews-v1 created
deployment.apps/reviews-v2 created
deployment.apps/reviews-v3 created
service/productpage created
serviceaccount/bookinfo-productpage created
deployment.apps/productpage-v1 created
```

The services have been created successfully, but most pods are still in the "ContainerCreating" state, which means they are starting up and pulling images. Only one pod is already running, indicating the deployment is in progress.

```
(base) PS D:\我的\大三\大三下\云计算导论\实验\石磊实验\Code\lab04\istio-1.22.3> kubectl get pods
>> kubectl get services
NAME                              READY   STATUS              RESTARTS      AGE
details-v1-666695dd7d-vlknf       0/1     ContainerCreating   0             16s
hello-node-64fc5894d-9nf8g        1/1     Running             2 (16m ago)   3d1h
productpage-v1-546779747f-br9t5   0/1     ContainerCreating   0             15s
ratings-v1-6db5b85498-7xfgj       0/1     ContainerCreating   0             16s
reviews-v1-7f79559d46-4qrzb       0/1     ContainerCreating   0             16s
reviews-v2-6f46b796b6-pft4f       0/1     ContainerCreating   0             16s
reviews-v3-5dbcb94d98-2xjwh       0/1     ContainerCreating   0             16s
NAME          TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
details       ClusterIP      10.111.105.28    <none>        9080/TCP         17s
hello-node    LoadBalancer   10.105.167.199   <pending>     8080:30893/TCP   3d1h
kubernetes    ClusterIP      10.96.0.1        <none>        443/TCP          3d1h
productpage   ClusterIP      10.110.190.138   <none>        9080/TCP         16s
ratings       ClusterIP      10.104.62.27     <none>        9080/TCP         16s
reviews       ClusterIP      10.106.76.33     <none>        9080/TCP         16s
```

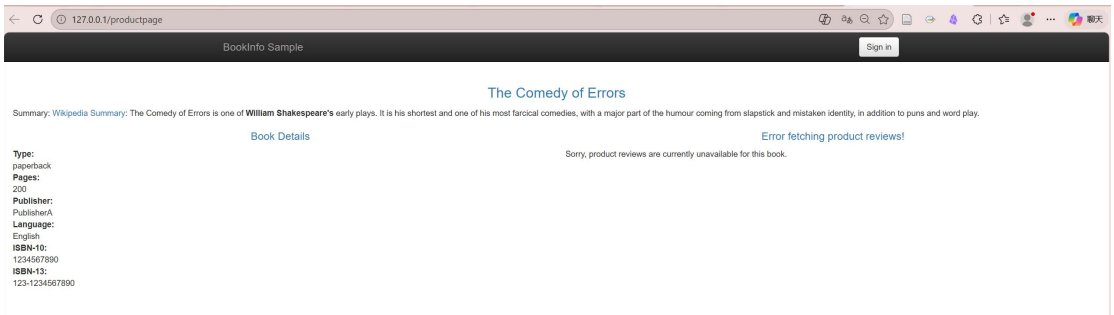
The request to the Bookinfo application is successful, and the returned page title confirms that the service is working correctly. This indicates that the microservices can communicate with each other and the application is running as expected.

```
(base) PS D:\我的\大三\大三下\云计算导论\实验\石磊实验\Code\lab04\istio-1.22.3> $pod = (kubectl ge
t pod -n default -l app=ratings --field-selector=status.phase=Running -o jsonpath="{.items[0].meta
data.name}").Trim()
>> $html = kubectl exec -n default $pod -- curl -sS productpage:9080/
>> $m = [regex]::Match($html, '<title>\s*(.*?)\s*</title>', 'IgnoreCase,Singleline')
>> $m.Groups[1].Value
Simple Bookstore App
```

The gateway configuration has been successfully applied, enabling external access to the Bookinfo application. The analysis output shows only informational warnings, mainly indicating that the namespace has not enabled automatic sidecar injection and that a service port naming convention is not fully compliant. These do not affect basic functionality but may impact advanced Istio features.

```
(base) PS D:\我的\大三\大三下\云计算导论\实验\石磊实验\Code\lab04\istio-1.22.3> kubectl apply -f s
amples/bookinfo/networking/bookinfo-gateway.yaml
gateway.networking.istio.io/bookinfo-gateway created
virtualservice.networking.istio.io/bookinfo created
(base) PS D:\我的\大三\大三下\云计算导论\实验\石磊实验\Code\lab04\istio-1.22.3> istioctl analyze
Info [IST0102] (Namespace default) The namespace is not enabled for Istio injection. Run 'kubectl
label namespace default istio-injection=enabled' to enable it, or 'kubectl label namespace default
 istio-injection=disabled' to explicitly mark it as not needing injection.
Info [IST0118] (Service default/hello-node) Port name  (port: 8080, targetPort: 8080) doesn't foll
ow the naming convention of Istio port.
```

The Bookinfo application is successfully accessed through the browser, and the product page is displayed correctly. This confirms that the service deployment, networking configuration, and external access are all functioning as expected.



四、Reflection

1. This experiment provides a clear and practical understanding of how a service mesh operates in a Kubernetes environment. Compared with traditional microservices communication, Istio introduces an additional abstraction layer through sidecar proxies, which significantly enhances traffic control and observability. The overall workflow becomes more structured, and complex functionalities can be implemented without modifying application code.

2. The installation and configuration process highlights the importance of environment consistency and dependency management. Small issues such as missing namespace labels or incorrect configurations can directly affect the behavior of the

system. This reinforces the need for careful verification at each step, especially when deploying distributed systems where errors are often not immediately visible.

3. The concept of sidecar injection is particularly insightful. It demonstrates how infrastructure-level capabilities can be decoupled from application logic, allowing developers to focus on business functionality while delegating concerns such as routing, monitoring, and security to the service mesh. This design reflects a more scalable and maintainable approach to modern system architecture.

4. The use of Gateway API for external access reveals a more standardized and flexible method for managing ingress traffic. Compared with traditional ingress configurations, it provides clearer abstraction and better extensibility. At the same time, it also introduces additional configuration complexity, which requires a deeper understanding of Kubernetes networking concepts.

5. The visualization through Kiali offers an intuitive perspective on service interactions, making abstract traffic flows easier to understand. Observing real-time request paths and dependencies helps identify potential bottlenecks and improves system transparency. This part of the experiment emphasizes the value of observability tools in managing and optimizing microservices systems.